

# Lab05 - BikeStation

---

## Général

- Auteurs: Fabien Léger & Aymeric Bonny
- Cours: PCO, HEIG-VD
- Data: 09.12.2025

## Problématique

Il était demandé dans ce labo de créer un système de vélos libres-services.

Les personnes peuvent prendre et mettre des vélos à des stations. Si la station est vide, ils attendent qu'un vélo soit déposé et vice-versa si la station est pleine.

Il faut ainsi mettre en place un moniteur de Mesa pour faire attendre les personnes avant qu'ils puissent prendre ou déposer un vélo dépendant de l'état de la station.

## Implémentation

### Person

Il a suffi de principalement suivre la donnée.

Plusieurs `if(stopRequested())` ont été ajoutés pour s'arrêter dans le cas où un `stopSimulation()` est arrivé. On peut ainsi à la fin de n'importe quel trajet sortir de notre boucle while.

```
// if a stop was requested, stop here
if (PcoThread::thisThread()->stopRequested())
    break;
```

Pour le site K, il paraissait logique de choisir un site différent de I ET de J. Nous avons donc une boucle un peu bizarre de la forme :

```
// we want K to be different from both I and J
while (siteK == currentSite)
    siteK = chooseOtherSite(homeSite);
```

Les deux fonctions à compléter `takeBikeFromSite()` et `depositBikeAtSite` appellent simplement `BikeStation`.

### Van

Pour la fonctionnalité du van, nous avons majoritairement suivi le pseudo-code proposé dans le pdf. Nous avons pris quelques libertés quand nous avons l'impression qu'elles étaient pertinentes, comme le minimum

de vélos à charger au moment de partir du dépôt, qui dépend maintenant aussi de la place restante dans le van. Une autre liberté que nous avons prise est de faire rentrer le van au dépôt quand la simulation s'arrête, pour que les employés aient la possibilité de se reposer confortablement. Le temps réglementaire de pause n'étant pas défini, nous avons choisi 0.5 seconde.

La plus grande difficulté était probablement d'interpréter correctement ce pseudo-code en fonction du squelette mis à notre disposition, comme la variable `a` que nous avons commencé par créer tel quel, avant de se rendre compte qu'elle représentait en réalité `cargo.size()`, ou encore `Vi` qui n'était pas une variable présente dans `van.cpp` mais qui était changé par les fonctions appelées.

Pour ce qui est de la sécurité, le van n'a pas fondamentalement besoin d'être protégé car il est déjà blindé (peinture verte).

BikeStation

Au niveau de l'implémentation, nous avons décidé de partir sur des queues de condition. Grâce à cela, on évite un nombre limite qu'on aurait avec une machine à ticket (même si celle-ci est très grande).

Nous avons décomposé le problème en différents attributs

| Variable       | Type                          | Description                                                                                                                                                                                        |
|----------------|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mutex          | PcoMutex                      | Protéger pour avoir uniquement une personne parcourant les <code>getBike()/putBike()/addBikes()/putBikes()</code> et ainsi protéger les différentes variables comme le demande un moniteur de Mesa |
| bikeGetPerType | Array de Queues de Conditions | Les personnes prennent les vélos selon le type (array du nombre de types) et FIFO (queue)                                                                                                          |
| bikePut        | Queue de Conditions           | Les personnes posent les vélos indépendamment du type selon FIFO (queue)                                                                                                                           |
| bikesPerType   | Array de Queues de conditions | Les vélos sont rangés suivant le type (array du nombre de types) et FIFO (queue)                                                                                                                   |
| stopped        | bool                          | Deviens true si <code>stopSimulation</code> est appelé pour renvoyer les threads entrant et libérer ceux coincés                                                                                   |

Il a été nécessaire d'avoir des queues pour chaque structure pour les raisons suivantes :

- `bikeGetPerType` : on prend un vélo suivant le premier arrivé à la station (en plus du type pour pas attendre pour rien)
- `bikePut` : on met un vélo suivant le premier arrivé à la station
- `bikesPerType` : Il est nécessaire de prendre les vélos avec `getBikes()` selon type et FIFO donc on doit avoir une queue

On a surtout suivi le code qui nous était donnée pour un moniteur de Mesa sinon. La seule différence est que nous n'avons pas eu besoin de variables pour compter, car toutes les conditions sont déjà dans nos queues et il est facile de savoir combien il y en a avec la fonction `size()`.

## Stop simulation

Comme dans le labo précédent, on demande aux threads de s'arrêter puis on appelle `bikeStation->ending()` pour toutes les stations pour empêcher les threads de rentrer à nouveau dans la classe, mais aussi de libérer ceux actuellement bloqués en attente d'un vélo ou d'une place.

## Tests

Tester `person.h` et `van.h` s'avère difficile, car ces classes sont privées. Il semble possible de mettre les tests en tant que friend mais nous nous sommes vite fait rendu compte de la complexité de tester ces classes. Il fallait créer des `BikeStation` valables, différents vélos, essayer différentes valeurs d'entrées.

Tester `BikeStations` aurait été le bienvenu, mais nous n'avons malheureusement pas vraiment eu le temps et il a été dit sur le Teams que ceux-ci n'étaient pas nécessaires. Nous avons donc préféré rendre le code propre plutôt que faire de nombreux tests.

Toutefois, tout semble fonctionner correctement suivant les indications de la donnée.

## Conclusion

Il n'y a pas vraiment eu de problèmes lors de ce labo. Le plus difficile a été de réfléchir aux bonnes structures pour faire fonctionner le moniteur de Mesa.

Pour des raisons qui nous sont inconnues, la GUI se relance de manière impromptue.